

The Stack ADT: Two Implementations, One Contract

CSCI 235 — Class #9

Tong Yi

Hunter College, CUNY

Tuesday, September 29, 2026

A Summary of Where We Have Been

Class	Topic	The idea to keep
1–3	C++ review, classes, memory	Memory from <i>new</i> must eventually be released with <i>delete</i> .
4–5	MiniVector, dynamic arrays	Growing an array means copying its existing elements into a larger array.
6	Abstract Data Types	The <i>contract</i> is separate from the <i>implementation</i> .
7	Nodes and linked lists	Start at <i>head</i> ; follow <i>next</i> links to reach other nodes.
8	Insert, delete, reverse	Change <i>head</i> for the first node; otherwise change the predecessor's <i>next</i> .
9 (today)	Stacks	One contract, two very different implementations.

Everything from Class 8 comes back today — not as review, but because a stack *is* those operations wearing a different name.

Your Questions From Thursday

Thank you for the slips from Thursday, 9/24.

- “*May there be a summary of what we learned in this class?*” — the slide you just saw. It will come back at the start of each class.
- “*I find `prev` confusing, could it be explained a bit more next class? And reversal.*” — the next three slides.
- “*Can we keep getting the slides during class?*” — yes. See the last slide.
- “*I’m a little bad at nodes but you explained it in a different way which I understood.*” — thank you for saying so. That is how I learn which explanation to keep.

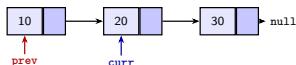
Why these slips matter

I cannot see confusion from the front of the room. A slip that says “`prev` is confusing” changes the next lecture. This one did.

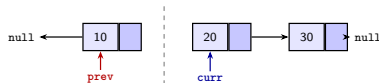
prev Is Confusing for a Real Reason

We used the name `prev` for **two different jobs** last class. Both pictures show 10 → 20 → 30 part way through.

In `insertAt` / `removeValue`



In `reverse`



One list. `prev` is simply the node *just before* `curr`. The chain is still intact.

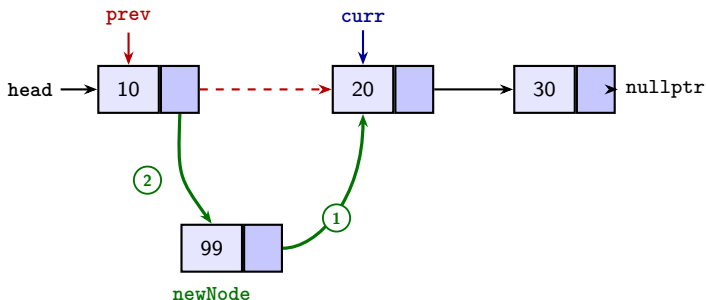
Two lists. `prev` heads the part *already reversed*; no link joins it to `curr`.

Same name, different job

In `insertAt`, `prev` is a *position* — it reaches the next field you must overwrite. In `reverse`, `prev` is an *accumulator* — it grows by one node each pass, and it is what the flipped link points back at. Read it as “the node before” in `reverse` and nothing makes sense: after the first flip, the dashed line is where “before” used to be.

prev as a *Position*: Why You Cannot Do Without It

Insert 99 between 10 and 20. The walk has stopped with prev on 10 and curr on 20.



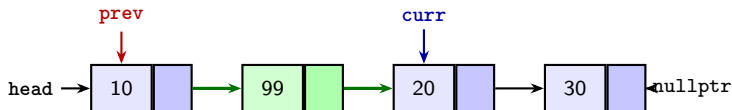
Why prev has to exist

Step ② writes into **node 10's next field**. That is the only field in the whole list that has to change — and curr cannot reach it. **In a singly linked list you can go forward, never backward**, so from node 20 there is no route back to node 10.

That is the entire reason for the second pointer. prev is not a trick; it is how you remember an address you can no longer look up.

prev as a *Position*: Why You Cannot Do Without It

Insert 99 between 10 and 20. The walk has stopped with prev on 10 and curr on 20.



10 -> 99 -> 20 -> 30 — one list again.
prev and curr never moved.

Why prev has to exist

Step ② writes into **node 10's next field**. That is the only field in the whole list that has to change — and curr cannot reach it. **In a singly linked list you can go forward, never backward**, so from node 20 there is no route back to node 10.

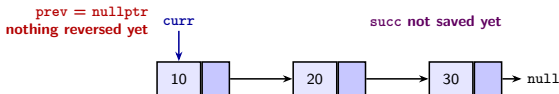
That is the entire reason for the second pointer. prev is not a trick; it is how you remember an address you can no longer look up.

prev as an *Accumulator*: Reversal, Slowly

Read prev not as “the node before” but as **the list I have finished reversing so far**. The marker follows the picture.

Pass	prev — reversed so far		curr — still to do	
	at	list from there	at	list from there
► start	nullptr	—	node 10	10 -> 20 -> 30
after 1	node 10	10	node 20	20 -> 30
after 2	node 20	20 -> 10	node 30	30
after 3	node 30	30 -> 20 -> 10	nullptr	—

The nodes **never move**; reversed links are drawn below the row.



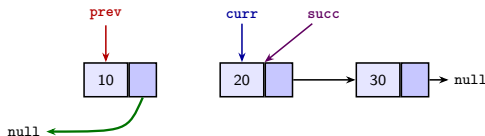
at is an arrow above the row; **list from there** is what you get by following the links. succ was saved *before* the flip destroyed curr->next, and the pass ends with curr = succ — which is why the two land on the same node. At the end head = prev; makes node 30 the new front.

prev as an *Accumulator*: Reversal, Slowly

Read prev not as “the node before” but as **the list I have finished reversing so far**. The marker follows the picture.

Pass	prev — reversed so far		curr — still to do	
	at	list from there	at	list from there
start	nullptr	—	node 10	10 -> 20 -> 30
▶ after 1	node 10	10	node 20	20 -> 30
after 2	node 20	20 -> 10	node 30	30
after 3	node 30	30 -> 20 -> 10	nullptr	—

The nodes **never move**; reversed links are drawn below the row.



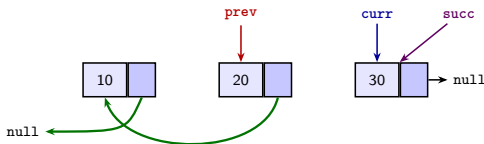
at is an arrow above the row; **list from there** is what you get by following the links. **succ** was saved *before* the flip destroyed **curr**→**next**, and the pass ends with **curr** = **succ** — which is why the two land on the same node. At the end **head** = **prev**; makes node 30 the new front.

prev as an *Accumulator*: Reversal, Slowly

Read prev not as “the node before” but as **the list I have finished reversing so far**. The marker follows the picture.

Pass	prev — reversed so far		curr — still to do	
	at	list from there	at	list from there
start	nullptr	—	node 10	10 -> 20 -> 30
after 1	node 10	10	node 20	20 -> 30
▶ after 2	node 20	20 -> 10	node 30	30
after 3	node 30	30 -> 20 -> 10	nullptr	—

The nodes **never move**; reversed links are drawn below the row.



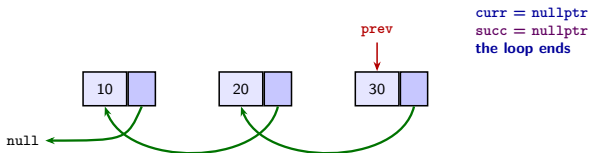
at is an arrow above the row; **list from there** is what you get by following the links. succ was saved *before* the flip destroyed curr->next, and the pass ends with curr = succ — which is why the two land on the same node. At the end head = prev; makes node 30 the new front.

prev as an *Accumulator*: Reversal, Slowly

Read prev not as “the node before” but as **the list I have finished reversing so far**. The marker follows the picture.

Pass	prev — reversed so far		curr — still to do	
	at	list from there	at	list from there
start	nullptr	—	node 10	10 -> 20 -> 30
after 1	node 10	10	node 20	20 -> 30
after 2	node 20	20 -> 10	node 30	30
► after 3	node 30	30 -> 20 -> 10	nullptr	—

The nodes **never move**; reversed links are drawn below the row.



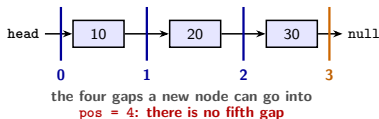
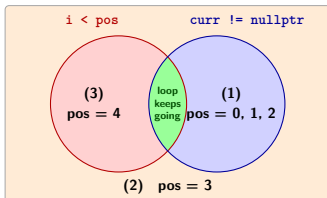
at is an arrow above the row; **list from there** is what you get by following the links. succ was saved *before* the flip destroyed curr->next, and the pass ends with curr = succ — which is why the two land on the same node. At the end head = prev; makes node 30 the new front.

“What Does `i == pos` Mean?”

Several of you asked. It is the most important line in `insertAt`, and it is easy to read past.

```
int i = 0; Node* prev = nullptr; Node* curr = head;
while (i < pos && curr != nullptr) {
    i++; prev = curr; curr = curr->next; // take one step
}
if (i == pos) { ... } // did we actually get there?
```

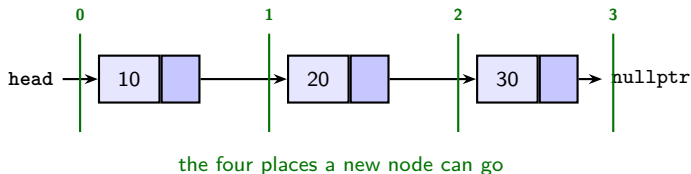
The loop continues while **both** conditions hold — the green overlap. When it stops, at least one condition is false, placing it in one of the other three regions, each its own colour.



- (1) `i == pos, curr != nullptr` — insert *before* an existing node.
- (2) `i == pos, curr == nullptr` — insert at the end; this is appending.
- (3) `i < pos, curr == nullptr` — past the end; return false.

Which Positions Are Even Legal?

Positions are counted from 0. For a list of length n , **insertion** accepts 0 through n — *inclusive*.



A distinction worth writing down

`getEntry(pos)` accepts $0 \dots n - 1$ — it must land *on* an existing node.

`insertAt(pos, v)` accepts $0 \dots n$ — it lands in a *gap*, and a list of length n has $n + 1$ gaps. Inserting at position n is appending.

insertAt: Every Case, One List

Insert 99 into 10 → 20 → 30 (length 3). Watch which pointer changes.

pos	prev	curr	Line that runs	Result
0	nullptr	node 10	head = newNode	99 → 10 → 20 → 30
1	node 10	node 20	prev->next = ...	10 → 99 → 20 → 30
3	node 30	nullptr	prev->next = ...	10 → 20 → 30 → 99
4	node 30	nullptr	none — i false, list unchanged stopped at 3	

Empty list, pos = 0: both prev and curr are nullptr, so head = newNode. It works, with no extra case.

Read the prev column, not the pos column

“Beginning”, “middle” and “end” are *your* words, not the code’s. The code asks one question: **is prev null?** Null means nothing is in front, so head must change. Otherwise some node’s next must change.

removeValue: The Same Three Cases

Remove from 10 -> 20 -> 30. The loop stops either **on** the target or at nullptr.

target	prev	curr	Line that runs	Result
10 (<i>first</i>)	nullptr	node 10	head = curr->next	20 -> 30
20 (<i>middle</i>)	node 10	node 20	prev->next = curr->next	10 -> 30
30 (<i>last</i>)	node 20	node 30	prev->next = curr->next	10 -> 20
99 (<i>absent</i>)	node 30	nullptr	none	false

Look again at the last node

Removing the **last** node needs *no special case*: prev->next = curr->next works because curr->next is already nullptr, so prev becomes the new last node automatically.

The **first** node is the only special one — and for the same reason as in insertAt: nothing points at it except head.

Two Shapes, Not Six

You met six cases across Class 8 — insert at the front, middle, back; remove the first, middle, last. There are really only **two**.

	<code>prev == nullptr</code>	<code>prev is a node</code>
What it means	<code>curr</code> is the first node, or the list is empty	some node sits in front of <code>curr</code>
What must change	<code>head</code>	<code>prev->next</code>
<code>insertAt</code>	<code>head = newNode</code>	<code>prev->next = newNode</code>
<code>removeValue</code>	<code>head = curr->next</code>	<code>prev->next = curr->next</code>

The question to ask every time

“What currently points at the node I am about to change?” If the answer is `head`, assign to `head`. If it is another node, assign to that node's `next`. Everything else — empty list, first, middle, last — follows from that one question. You do not need to memorise six cases.

Prediction 1: What Comes Out First?

You have a stack of `int`. You run:

```
push(10); push(20); push(30); pop();
```

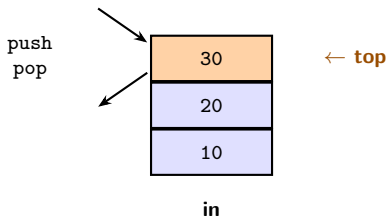
What value does that `pop()` remove?

- (A) 10 — it went in first
- (B) 30 — it went in last
- (C) 20 — the middle one
- (D) Whichever is cheapest to reach

Vote on your phone: scan the QR code or use the Brightspace link.

Prediction 1: Reveal

(B) 30. The implementation should also make the top cheap to reach.



LIFO: Last In, First Out

A stack only ever lets you touch **one end**. Anything buried underneath is unreachable until what is above it comes off.

Why would anyone accept such a limited structure? **Because a restriction you can rely on is worth more than freedom you do not need.** A stack can support its basic operations in $O(1)$ because it never needs to search for an arbitrary position.

The Stack ADT: The Whole Contract

Recall Class 6: an ADT says *what*, never *how*. Here is all of it.

Operation	What it promises	If the stack is empty
push(v)	v becomes the new top	—
pop()	removes the top item	invalid operation — behavior depends on the interface
top()	reads the top item, removes nothing	invalid operation — behavior depends on the interface
empty()	true when there is nothing in it	returns true
size()	how many items are in it	returns 0

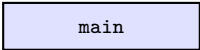
Notice what is *not* in the contract

No indexing. No searching. No “give me the third one.” No iteration. If your algorithm needs those, a stack is the wrong container — and that is a design answer, not a limitation you should work around.

You Have Been Using a Stack All Semester

Ordinary C++ implementations use a **call stack** to manage active function calls.

`main` starts running.
Its locals live in that box.



`main`

Why a stack is the natural model

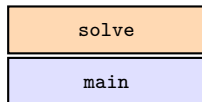
Calls finish in the *opposite* order they were made. The most recently called function is always the next one to return. That is LIFO, and this LIFO behavior is naturally modeled by a stack.

You Have Been Using a Stack All Semester

Ordinary C++ implementations use a **call stack** to manage active function calls.

`main` calls `solve`.

A new frame is **pushed** on top.

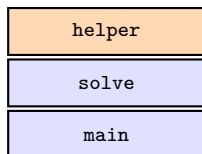


Why a stack is the natural model

Calls finish in the *opposite* order they were made. The most recently called function is always the next one to return. That is LIFO, and this LIFO behavior is naturally modeled by a stack.

You Have Been Using a Stack All Semester

Ordinary C++ implementations use a **call stack** to manage active function calls.



solve calls helper.
Pushed again. solve is now *buried* — it cannot continue until helper returns.

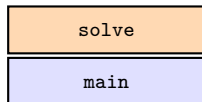
Why a stack is the natural model

Calls finish in the *opposite* order they were made. The most recently called function is always the next one to return. That is LIFO, and this LIFO behavior is naturally modeled by a stack.

You Have Been Using a Stack All Semester

Ordinary C++ implementations use a **call stack** to manage active function calls.

helper returns.
Its frame is **popped**, and
its locals are gone.



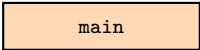
Why a stack is the natural model

Calls finish in the *opposite* order they were made. The most recently called function is always the next one to return. That is LIFO, and this LIFO behavior is naturally modeled by a stack.

You Have Been Using a Stack All Semester

Ordinary C++ implementations use a **call stack** to manage active function calls.

solve returns. Popped.
Back in `main`, exactly
where it left off.



`main`

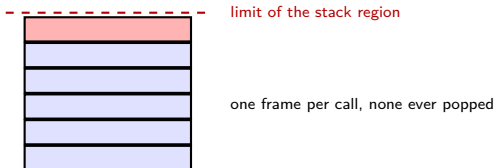
Why a stack is the natural model

Calls finish in the *opposite* order they were made. The most recently called function is always the next one to return. That is LIFO, and this LIFO behavior is naturally modeled by a stack.

What “Stack Overflow” Actually Means

The call stack has a finite size limit. If recursive calls keep adding frames without returning, the call stack eventually runs out of space.

```
int countdown(int n) {  
    return countdown(n - 1); // no base case: never returns  
}
```

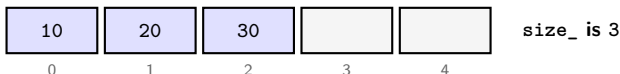


The name is literal

A “stack overflow” is a stack that overflowed. A common cause is recursion without a working base case, or recursion that becomes too deep — which we take apart in **Class 11**.

Prediction 2: Where Does the Top Live?

You store the items in a plain array and keep one integer, `size_`, holding how many items are in the stack.



Which expression reads the top item?

- (A) `data_[size_]`
- (B) `data_[size_ - 1]`
- (C) `data_[0]`
- (D) Either (A) or (B), depending on how you push

Vote on your phone: scan the QR code or use the Brightspace link.

Prediction 2: Reveal

(B) `data_[size_ - 1]` — and (D) confuses two different meanings for the stored integer. A count uses `data_[size_ - 1]`; a top index uses `data_[top_]`.

	<code>size_ = count</code>	<code>top_ = index of top</code>
Empty is written	<code>size_ == 0</code>	<code>top_ == -1</code>
Top item is	<code>data_[size_ - 1]</code>	<code>data_[top_]</code>
push	<code>data_[size_++] = v;</code>	<code>data_[++top_] = v;</code>
<code>size()</code> returns	<code>size_</code>	<code>top_ + 1</code>

Both are correct. Pick one and never mix them.

We use the `size_` column, for the same reason `MiniVector` did: `size_` is the number the ADT already has to report, and the empty case is 0 rather than a magic `-1`.

Nearly every off-by-one bug in a stack comes from writing one method in the left column and the next method in the right one.

The Array Stack — Your Turn

The declarations are given. Write the three marked bodies.

```
class ArrayStack {
public:
    bool push(int value); // false if full
    bool pop();           // false if empty
    int top() const;      // caller must check empty() first
    bool empty() const { return size_ == 0; }
    int size() const { return size_; }
private:
    static const int CAPACITY = 100;
    int data_[CAPACITY];
    int size_ = 0;
};

bool ArrayStack::push(int value) { STEP 1 }
bool ArrayStack::pop()           { STEP 2 }
int ArrayStack::top() const      { STEP 3 }
```

Before you write

For each operation, ask what can go wrong and what the contract says should happen. `push()` and `pop()` handle failure themselves; `top()` has the precondition `!empty()`.

The Array Stack — Your Turn

The declarations are given. Write the three marked bodies.

```
bool ArrayStack::push(int value) {  
    if (size_ == CAPACITY) { return false; } // guard  
    data_[size_] = value;                    // write  
    size_++;                                // then count  
    return true;  
}  
  
bool ArrayStack::pop() {  
    if (size_ == 0) { return false; } // guard  
    size_--;                            // that is the whole deletion  
    return true;  
}  
  
int ArrayStack::top() const {  
    return data_[size_ - 1]; // precondition: !empty()  
}
```

pop really is just size_--

The old value is still in `data_`, but now *outside* the stack, and the next push overwrites it. No dynamic memory was allocated for that element, so nothing needs to be freed when it is popped.

The Honest Weakness of the Array Stack

What is good

Every operation is $O(1)$. The items sit next to each other in memory, giving the array stack good memory locality. No dynamic allocation by the stack itself, so its storage cannot leak.

What is not

CAPACITY is decided when you write the class, not when you use it. Ask for item 101 and `push` returns `false` — on a stack that is logically nowhere near full.

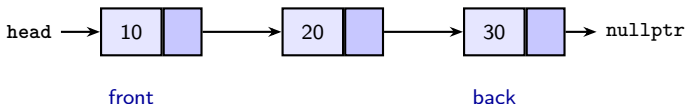
You already know the fix, and its price

Grow the array like `MiniVector` did: allocate a bigger one, copy everything across, delete the old. That removes the fixed capacity, but one `push` in a while stops being $O(1)$ — it becomes $O(n)$, because all existing elements must be copied or moved into a larger array. With geometric growth, however, `push` is still **amortized** $O(1)$.

So: what if we want a stack with **no fixed capacity** that **never copies existing elements**?

Prediction 3: Which End Should Be the Top?

You store the items in a singly linked list, and must choose which end is the top.



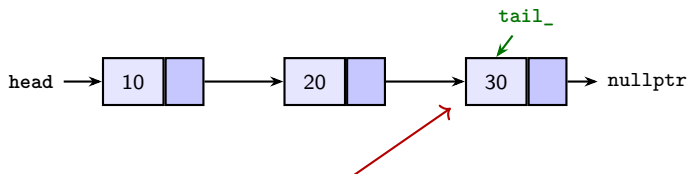
Put the top at the back, and keep a `tail_` pointer so you can reach it in $O(1)$. Does `pop()` then run in $O(1)$?

- (A) Yes — `tail_` points straight at it
- (B) No — it is still $O(n)$
- (C) Only when the list has at least two nodes

Vote on your phone: scan the QR code or use the Brightspace link.

Prediction 3: Reveal — and This Is Thursday's Lesson

(B) No. It is still $O(n)$, tail pointer or not.



to delete node 30, node 20 must stop pointing at it —
and from `tail_` there is no way back to 20

This is exactly what `prev` was for

`tail_` gets you to the last node in one step. It does not get you to the node **before** it — and that is the one whose `next` must change. You would have to walk from `head` again: $O(n)$.

So the top goes at the **front**, where `head` already is. Hold on to this: on Thursday a `tail_` pointer will be exactly the right answer — because a queue only ever *adds* at the back.

The Linked Stack: You Wrote This Last Class

Top at the front. Now look at what push and pop turn into.

Class 8: prepend

```
void prepend(Node*& head,
             int value) {
    Node* newNode =
        new Node{value, head};
    head = newNode;
}
```

Class 8: removeFront

```
void removeFront(Node*& head) {
    if (head == nullptr) return;
    Node* toDelete = head;
    head = head->next;
    delete toDelete;
}
```

Today: push

```
bool LinkedStack::push(int v) {
    Node* newNode =
        new Node{v, head_};
    head_ = newNode;
    size_++;
    return true;
}
```

Today: pop

```
bool LinkedStack::pop() {
    if (head_ == nullptr)
        return false;
    Node* toDelete = head_;
    head_ = head_>next;
    delete toDelete;
    size_--;
    return true;
}
```

Same code, new name

Nothing was invented today. A linked stack is a linked list *forbidden* to touch anything but the front, that is, put the top at the front — and that restriction is what makes operations push, pop, top, empty, and size $O(1)$.

top(), and the Two Things That Bite

```
int LinkedStack::top() const {  
    return head_>data; // precondition: !empty()  
}  
  
LinkedStack::~~LinkedStack() {  
    while (head_ != nullptr) { // free every node  
        Node* toDelete = head_;  
        head_ = head_>next;  
        delete toDelete;  
    }  
}
```

Two

- (1) `top()` on an empty stack dereferences `nullptr`, causing undefined behavior; the program may crash. The contract says the caller must check `empty()` first — so *check it*.
- (2) The array stack cannot leak from its stack storage. This one allocates with `new` on every `push`, so it needs a destructor — and Valgrind will find it if you forget.

Two Implementations, One Contract

	Array-based	Linked
push / pop / top	$O(1)$	$O(1)$
Capacity	fixed at compile time	limited only by memory
Memory per item	just the item	the item <i>and</i> a pointer
Dynamic allocation	none	one new per push
Can leak from stack storage itself?	no	yes, without a destructor
Memory locality	better — items are adjacent	worse — nodes may be scattered

The ADT's whole point, demonstrated

Algorithms that use the stack do not have to change: they still use push, pop, top, empty and size. The contract is identical; only the representation and the costs differ.

That is what Class 6 was claiming, and now we can see it in a concrete example.

Vector and Stack Share Two Operations

You have already written `push_back` and `pop_back` in `MiniVector`. A stack's `push` and `pop` do the same two things at the same end.

What you want	vector	Stack
add at the end	<code>push_back(v)</code>	<code>push(v)</code>
remove from the end	<code>pop_back()</code>	<code>pop()</code>
look at the end	<code>back()</code>	<code>top()</code>
how many	<code>size()</code>	<code>size()</code>
is it empty	<code>empty()</code>	<code>empty()</code>
element number <i>i</i>	<code>v[i]</code> , in $O(1)$	not offered
insert in the middle	<code>insert(...)</code>	not offered
erase anywhere	<code>erase(...)</code>	not offered

A vector can implement the stack ADT

`push_back` then `pop_back` gives you last-in-first-out. These five stack operations—`push`, `pop`, `top`, `empty`, and `size`—correspond to operations a vector provides, which raises the obvious question: why have a stack at all? Two answers, and the second is the surprising one.

Why Does vector Use an Array?

First answer: the restriction is the point. **Second answer:** what an ADT promises constrains which implementations are appropriate.

	The promise that costs you	A natural representation
vector	contiguous storage and $O(1)$ indexed access, $v[i]$	a dynamic array naturally provides both
Stack	a much weaker promise: you may access only the top	the interface does not require indexed access, so the implementation has more freedom — an array <i>or</i> linked nodes
Linked list	$O(1)$ insertion/removal at the front, insertion after a known node, or removal of its successor. Finding the node can take $O(n)$.	nodes: a value and a pointer to the next one

The idea to keep

The operations promised by an ADT constrain which implementations are **appropriate**. Linked nodes could not serve $v[i]$ in $O(1)$ — reaching element i means walking i links; and an array cannot cheaply insert in the middle, because every element after the gap has to shift.

Which One Would You Reach For?

	Reach for it when	You have already seen
vector	you look things up by position , and mostly add at the end	Connect 4 : the board is read by row and column. Exercise 1 : shapes sorted by area — sorting compares <code>a[i]</code> with <code>a[j]</code>
Stack	you always need the most recent unfinished thing	The call stack : the newest frame returns first. Matching brackets : the last (seen is the one the next) must close
Linked list	you often insert after a known node or remove its successor, and never ask for element <code>i</code>	Next class : a queue. Project 2 : the path out of the maze, each cell linked to the one before it

A linked list in the wild: a playlist

Suppose a playlist interface mainly supports moving through songs sequentially, inserting the next song, and removing one. Moving to the next song follows a link. Inserting or removing the next song changes only a few links. **If the interface does not require fast random access by position, linked nodes can be a reasonable representation.**

Prediction 4: Which One Is Unbalanced?

Your compiler checks this in every file you write.

(A) { [()] } (B) { [(]) } (C) { [()] }

Which are *not* balanced?

- (A) Only (B)
- (B) Only (C)
- (C) Both (B) and (C)
- (D) All three are fine — the counts match

Second question, and the harder one: **counting** brackets is not enough. Why not?

Vote on your phone: scan the QR code or use the Brightspace link.

Prediction 4: Reveal

(C) **Both.** And counting fails because it ignores *order*.

	Counts	Verdict
(A)	3 open, 3 close	balanced
(B)	3 open, 3 close	broken —] requires [, but the most recent unmatched opening bracket is (
(C)	3 open, 2 close	broken — { is never closed

Why a stack is the natural tool

When you meet a closing bracket, the only opening bracket it may match is the **most recent unmatched one**. “Most recent” is the word that decides the data structure.

Tracing the Bracket Matcher

Input: { [(]) } — push every opening bracket, and on every closing bracket check the top. *For this example, imagine the same stack stores char instead of int. The stack operations and the algorithm are unchanged.*

{ [(]) }

the stack

{ opens.
push it.

{

And at the very end

If the string runs out while the stack is **not empty**, some opening bracket was never closed — case (C). Both failures, one algorithm.

Tracing the Bracket Matcher

Input: { [(]) } — push every opening bracket, and on every closing bracket check the top. *For this example, imagine the same stack stores char instead of int. The stack operations and the algorithm are unchanged.*

{ [(]) }

the stack

[opens.
push it.



And at the very end

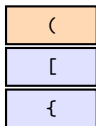
If the string runs out while the stack is **not empty**, some opening bracket was never closed — case (C). Both failures, one algorithm.

Tracing the Bracket Matcher

Input: { [(]) } — push every opening bracket, and on every closing bracket check the top. *For this example, imagine the same stack stores char instead of int. The stack operations and the algorithm are unchanged.*

{ [(]) }

the stack



(opens.
push it.

And at the very end

If the string runs out while the stack is **not empty**, some opening bracket was never closed — case (C). Both failures, one algorithm.

Tracing the Bracket Matcher

Input: { [(]) } — push every opening bracket, and on every closing bracket check the top. *For this example, imagine the same stack stores char instead of int. The stack operations and the algorithm are unchanged.*

{ [(]) }

the stack



] closes.

Look at the top: (.

Mismatch — reject.

The counts would never have caught this.

And at the very end

If the string runs out while the stack is **not empty**, some opening bracket was never closed — case (C). Both failures, one algorithm.

The Bracket Matcher, Complete

```
bool balanced(const std::string& text) {
    LinkedStack s;
    for (char c : text) {
        if (c == '(' || c == '[' || c == '{') {
            s.push(c);                // remember it
        } else if (c == ')' || c == ']' || c == '}') {
            if (s.empty()) { return false; } // nothing is open
            char open = s.top();
            s.pop();
            if (!matches(open, c)) { return false; }
        }
    }
    return s.empty();    // anything left over is unclosed
}
```

Three lines carry the whole idea

`s.push(c)` remembers an obligation. `s.top()` asks which obligation is most urgent. `return s.empty()` asks whether any are left.

*`top()` does not check whether the stack is empty — the precondition comment is a note to the reader, not code that runs. Checking `empty()` first is the caller's job, and *if* `(s.empty())` above is that check.*

Where Else Stacks Show Up

Undo (Ctrl+Z)	Every action is pushed. Undo pops the most recent one. This is why you cannot undo the third-to-last action without undoing the two after it.
The back button	Each page you visit is pushed; Back pops.
Function calls	The call stack — the one your program is using right now.
Reversing anything	Push all of it, then pop all of it. LIFO naturally produces the elements in reverse order.
Depth-first search	Class 11–12, Project 2: explore as deep as you can, and when you hit a dead end, <i>pop back</i> to the most recent junction with an untried direction.

The pattern under all five

Every one of them needs to come back to the **most recent** unfinished thing. When you catch yourself saying “most recent” about a problem, you have found a stack.

Thursday: Change One Word

A stack serves the **most recent** arrival first. What if you serve the **earliest** one instead?

	Stack (today)	Queue (Thursday)
Rule	LIFO — last in, first out	FIFO — first in, first out
Everyday picture	a pile of plates	the line at the registrar
Add at	the top	the back
Remove from	the top	the front
Ends in use	one	two

Why that one word costs you so much

A stack touches one end, so `head_` alone is enough. A queue touches *both* ends — so it needs a second pointer, and the array version has to solve a problem a stack never had: what happens when the back runs off the end of the array.

And the tail pointer that was useless today becomes the whole trick on Thursday.

Slides, Notes, and Pacing

You asked, and this is a fair request.

You asked	Answer
Can we have the slides <i>during</i> class?	Yes. From now on the PDF goes up on Brightspace and the course website before class , so you can follow along and annotate rather than copy.
The slides switch too fast while I take notes	Tell me in the moment — raise a hand, or call out “back.” I would much rather re-show a slide than lose you on it.

A suggestion about notes

If the slides are already in front of you, do not copy them. Write down the things that are *not* on them: the question you almost asked, the step that surprised you, the reason a line is in that order. That is the part you will need in November.

What's Due

- **Exercise 4** (Linked List Operations) — due **Thursday, October 1**.
- **Project 1B** — due **Thursday, October 1**.
- **Exercise 5** (Stack & Queue) — posted Thursday, due **Thursday, October 8**.
- **Project 2** (Maze Solver) — posted Thursday. It uses a stack. You now know why.

Before Thursday

Read **Chapter 6** (the ADT stack) and **Chapter 7** (stack implementations). And if any part of prev still does not sit right, put that on a slip — we are not finished with it.

Key Takeaways

- ❶ A stack is defined by a **restriction**: one end only, last in, first out. The restriction makes $O(1)$ basic stack operations possible.
- ❷ **prev has two different jobs**. A position in `insertAt`; an accumulator in `reverse`. Ask which one you are looking at.
- ❸ A **tail pointer does not help you delete** from the back of a singly linked list — you cannot reach the node before it.
- ❹ The linked stack is Class 8's `prepend` and `removeFront`, renamed.
- ❺ **One contract, two implementations**. An algorithm using the stack does not depend on how the stack stores its elements.

Questions?

Please fill out today's lecture slip.

Tell me what did not land. It changes the next class — three of today's slides exist because of Thursday's slips.

Backup Slides

Backup: The Linked Stack — the Class

```
class LinkedStack {
public:
    LinkedStack() = default;
    ~LinkedStack();
    bool push(int value);
    bool pop();
    int top() const; // precondition: !empty()
    bool empty() const { return head_ == nullptr; }
    int size() const { return size_; }
private:
    struct Node { int data; Node* next; };
    Node* head_ = nullptr;
    int size_ = 0;
};
```

Do not copy objects of this simplified class. The compiler-generated copy operations copy `head_` and `size_`, but do not duplicate the nodes. Both objects would own the same nodes. Safe copying requires custom copy operations.

Backup: The Linked Stack — the Methods

```
bool LinkedStack::push(int value) {
    Node* newNode = new Node{value, head_};
    head_ = newNode;
    size_++;
    return true;
}

bool LinkedStack::pop() {
    if (head_ == nullptr) { return false; }
    Node* toDelete = head_;
    head_ = head_>next;
    delete toDelete;
    size_--;
    return true;
}

int LinkedStack::top() const { return head_>data; }

LinkedStack::~LinkedStack() {
    while (head_ != nullptr) {
        Node* toDelete = head_;
        head_ = head_>next;
        delete toDelete;
    }
}
```

Backup: matches(), and a Cleaner Version

```
bool matches(char open, char close) {  
    return (open == '(' && close == ')')  
        || (open == '[' && close == ']')  
        || (open == '{' && close == '}');  
}
```

A common alternative: push the *expected closing bracket* instead of the opening one. Then the check is a plain ==.

```
if (c == '(') { s.push(')'); }  
else if (c == '[') { s.push(']'); }  
else if (c == '{') { s.push('}'); }  
else if (c == ')' || c == ']' || c == '}') {  
    if (s.empty() || s.top() != c) { return false; }  
    s.pop();  
}
```

Same algorithm. It just moves the decision to the moment you have the most information.

Backup: Reversing With a Stack

Class 8 reversed a list by re-pointing links. A stack can produce the elements in reverse order without modifying the original structure:

```
void printReversed(Node* head) {  
    LinkedStack s;  
    for (Node* curr = head; curr != nullptr; curr = curr->next) {  
        s.push(curr->data);    // all of it goes in  
    }  
    while (!s.empty()) {  
        std::cout << s.top() << " ";  
        s.pop();                // and comes out backwards  
    }  
}
```

The trade you just made

Class 8's reverse was $O(n)$ time and $O(1)$ space. This is $O(n)$ time and $O(n)$ space — the stack holds a copy of every element.

It does not modify the original list, and it is much harder to get wrong. Both are reasonable answers; they are answers to slightly different questions.